ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

BÁO CÁO ĐỒ ÁN MÔN HỌC

IT4863 – Tìm kiếm thông tin

MÁY TÌM KIẾM CHO MIỀN `HUST.EDU.VN`

Thu thập dữ liệu phân tán và xếp hạng kết quả trên Lucene

Giảng viên hướng dẫn: **TS. Nguyễn Bá Ngọc**

Nhóm thực hiện: **Tìm là thấy**

Họ và tên	MSHV
Nguyễn Nam Hán	20252095M
Nguyễn Phúc Nguyên	20261222M
Lê Văn Đức	20261233M

Hà Nội, học kỳ 2026.1A

Tóm tắt

Báo cáo trình bày máy tìm kiếm cho các website thuộc Đại học Bách khoa Hà Nội (`*.hust.edu.vn`) do nhóm *Tìm là thấy* xây dựng trong môn IT4863. Hệ thống gồm hai phần dùng chung một cơ sở dữ liệu MongoDB.

1. **Bộ thu thập dữ liệu (crawler)** gồm năm dịch vụ nhỏ trao đổi với nhau qua HTTP/JSON: `mediator` (Python, hàng đợi SQLite, giãn cách theo host, crawl lại định kỳ, phát hiện bài mới qua sitemap và trang danh mục), `downloader` (Node.js, Chromium headless qua Puppeteer, chạy bốn bản), `reader` (Java, jsoup và Apache Tika), `storage` (Spring Boot) và MongoDB. Crawler đã thu thập 28 168 trang web từ 95 tên miền con, 4 690 tài liệu (chủ yếu là PDF), 9 614 ảnh và 84 673 cạnh liên kết.
2. **Máy tìm kiếm** xây trên Apache Lucene 10. Văn bản tiếng Việt được phân tích thành unigram âm tiết, bigram và bản bỏ dấu. Việc xếp hạng chia hai bước: BM25 nhiều trường để lấy ứng viên, sau đó xếp hạng lại theo cụm từ và PageRank. Trang rác được phát hiện bằng độ đo "độ hiếm" với ngưỡng tự chọn theo thung lũng của histogram; trang trùng nội dung được loại bằng mã băm SHA-1. Bốn cấu hình xếp hạng A–D bật dần từng kỹ thuật, đi kèm công cụ đánh giá offline bằng pooling, P@10 và nDCG@10.

Cả hệ thống chạy được bằng một lệnh `docker compose up`. Giao diện web cho phép theo dõi tiến độ crawl, tìm kiếm và thêm url cần index ngay.

Mục lục

Tóm tắt **i**

1 Giới thiệu **1**
1.1 Bài toán . . . 1
1.2 Mục tiêu . . . 1
1.3 Cấu trúc báo cáo . . . 1

2 Kiến trúc tổng thể **3**
2.1 Sơ đồ hệ thống . . . 3
2.2 Các dịch vụ . . . 4
2.3 Mô hình dữ liệu . . . 4

3 Thu thập dữ liệu **6**
3.1 Dịch vụ mediator . . . 6
3.1.1 Vai trò . . . 6
3.1.2 Chuẩn hoá url . . . 6
3.1.3 Phân loại url và chu kỳ crawl lại . . . 7
3.1.4 Hàng đợi bền vững trên SQLite . . . 7
3.1.5 Luồng giao url . . . 8
3.1.6 Duy trì dữ liệu và tải bài mới . . . 9
3.1.7 API và cấu trúc mã . . . 10
3.2 Dịch vụ downloader . . . 11
3.2.1 Tải trang web bằng trình duyệt headless . . . 11
3.2.2 Tải tài liệu và ảnh . . . 11
3.3 Dịch vụ reader . . . 11
3.3.1 Lấy tiêu đề . . . 12
3.3.2 Tìm khối nội dung chính . . . 12
3.3.3 Liên kết và văn bản liên kết . . . 12
3.3.4 Đọc tài liệu và ảnh . . . 13
3.3.5 Chuyển tiếp . . . 13
3.4 Dịch vụ storage . . . 13
3.5 Nạp url mầm và theo dõi . . . 14

3.6 Kết quả thu thập 14

4 Máy tìm kiếm 16
4.1 Tổng quan 16
4.2 Phân tích văn bản tiếng Việt 17
4.2.1 Chuẩn hoá Unicode 17
4.2.2 Tách từ: unigram âm tiết cộng bigram 17
4.2.3 Bản bỏ dấu 18
4.3 Lược đồ chỉ mục 18
4.4 BM25 theo trường 19
4.5 PageRank 19
4.5.1 Mô hình 19
4.5.2 Cài đặt 19
4.5.3 Đưa vào điểm xếp hạng 20
4.6 Phát hiện trang rác bằng độ hiếm 20
4.6.1 Quan sát 20
4.6.2 Độ đo 20
4.6.3 Chọn ngưỡng tự động 20
4.7 Loại trang trùng nội dung 21
4.8 Quy trình dựng chỉ mục 21
4.9 Truy vấn hai bước 22
4.9.1 Bước 1: lấy ứng viên bằng BM25 nhiều trường 22
4.9.2 Bước 2: xếp hạng lại 23
4.9.3 Bốn profile xếp hạng 23
4.9.4 Snippet 24
4.9.5 Ví dụ 24
4.10 API tìm kiếm 24

5 Đánh giá 26
5.1 Mục tiêu và dữ liệu 26
5.2 Quy trình 26
5.3 Độ đo 26
5.4 Kết quả 27

6 Triển khai và kiểm thử 28
6.1 Triển khai 28
6.2 Kiểm thử 28

7 Hạn chế, hướng phát triển và kết luận 30
7.1 Hạn chế 30
7.2 Hướng phát triển 31

7.3 Kết luận 31

Tài liệu tham khảo **32**

Danh sách hình vẽ

2.1 Kiến trúc tổng thể của hệ thống 3

4.1 Hai quy trình của dịch vụ search 16

Danh sách bảng

2.1 Các dịch vụ và công nghệ 4
2.2 Các collection trong MongoDB 5

3.1 Phân loại url và chu kỳ kiểm tra lại 7
3.2 Số liệu dữ liệu thu thập được 14
3.3 Các tên miền con có nhiều trang nhất 15

4.1 Các analyzer, ví dụ với “Bách Khoa Hà Nội” 17
4.2 Các trường của tài liệu Lucene 18
4.3 Trọng số trường ở bước lấy ứng viên 22
4.4 Bốn profile xếp hạng 23
4.5 Các endpoint của dịch vụ search 24

5.1 Kết quả đánh giá offline trên 25 truy vấn (sẽ điền sau khi gán nhãn) . . . 27

6.1 Kiểm thử tự động 29

Chương 1

Giới thiệu

1.1 Bài toán

Thông tin của Đại học Bách khoa Hà Nội nằm rải rác trên gần một trăm website: cổng chính `hust.edu.vn`, website của các trường, khoa, viện (`soict`, `seee`, `bme`, `bktextile`…), thư viện, phòng ban và các trang dự án. Những thông tin sinh viên hay cần như học phí, lịch thi, quy chế đào tạo thường nằm trong các tệp PDF, DOCX đính kèm chứ không phải trong trang web. Các công cụ tìm kiếm tổng quát trả về lẫn cả website ngoài trường và ít khi đánh chỉ mục nội dung tệp đính kèm.

Đồ án xây dựng một máy tìm kiếm riêng cho miền `hust.edu.vn` và làm đủ các khâu của một hệ tìm kiếm web: thu thập, bóc nội dung, lưu trữ, đánh chỉ mục, xếp hạng và đánh giá.

1.2 Mục tiêu

- Tự động thu thập trang web, tài liệu và ảnh thuộc `*.hust.edu.vn`, mở rộng theo liên kết, không gửi yêu cầu dồn dập vào một máy chủ (giới hạn tốc độ theo host) và tự cập nhật định kỳ.
- Lấy được phần nội dung chính của trang, bỏ menu và chân trang; đọc được chữ trong PDF, DOCX, XLSX.
- Lưu nội dung và đồ thị liên kết vào MongoDB theo một lược đồ thống nhất.
- Tìm kiếm tiếng Việt chấp nhận cả truy vấn có dấu và không dấu, có dùng tín hiệu liên kết, lọc trang rác và trang trùng.
- Đo được mỗi kỹ thuật xếp hạng đóng góp bao nhiêu bằng đánh giá offline.

1.3 Cấu trúc báo cáo

Chương 2 nói về kiến trúc chung. Chương 3 trình bày bộ thu thập dữ liệu và số liệu thu được. Chương 4 trình bày máy tìm kiếm: phân tích văn bản, chỉ mục, BM25, PageRank,

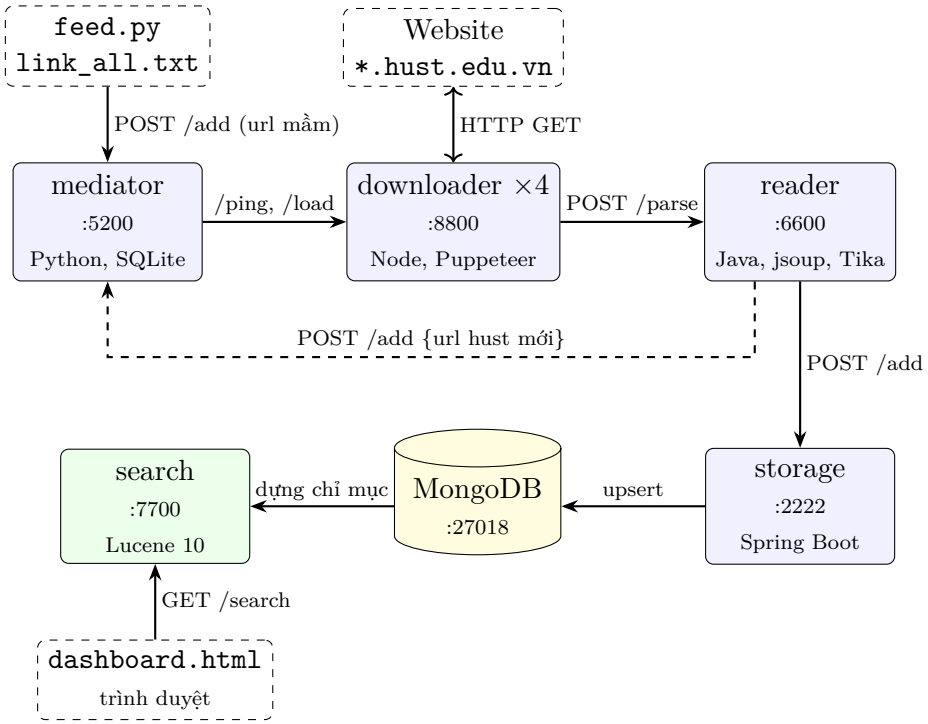
lọc rác, loại trang trùng và truy vấn hai bước. Chương 5 nói về cách đánh giá, Chương 6 về triển khai và kiểm thử, Chương 7 về hạn chế, hướng phát triển và kết luận.

Chương 2

Kiến trúc tổng thể

2.1 Sơ đồ hệ thống

Hệ thống gồm sáu dịch vụ. Mỗi dịch vụ có một thư mục và một `Dockerfile` riêng, trao đổi với nhau bằng HTTP và JSON (Hình 2.1). Dữ liệu đi theo một chiều: `mediator` → `downloader` → `reader` → `storage` → MongoDB. Vòng lặp crawl khép lại ở chỗ `reader` gửi các url `hust.edu.vn` mới tìm thấy về lại `mediator`. Dịch vụ `search` đọc MongoDB để dựng chỉ mục Lucene và trả lời truy vấn.



Hình 2.1: Kiến trúc tổng thể của hệ thống

Bảng 2.1: Các dịch vụ và công nghệ

Dịch vụ (thư mục)	Vai trò, công nghệ	Cổng
`mediator`	Giữ hàng đợi url (SQLite), giao url cho các bản downloader theo vòng, giãn cách theo host, hẹn lịch crawl lại, phát hiện bài mới. Python 3.13, `asyncio`, `aiohttp`.	5200
`downloader`	Dùng Chromium headless (Puppeteer) cho trang web, `fetch` cho tài liệu và ảnh. Mỗi bản chỉ tải một url mỗi lúc. Node.js 24.	8800
`reader`	Lấy tiêu đề, khối nội dung chính, liên kết; đọc chữ trong tệp bằng Apache Tika; chuyển kết quả sang `storage` và `mediator`. Java 21.	6600
`storage`	Kiểm tra và ghi bản ghi vào `html`/`other`/`picture`, ghi cạnh vào `graph`. Java 21, Spring Boot 3.3.	2222
`search`	Dựng chỉ mục, xếp hạng, API `/search`, phục vụ giao diện web. Java 21, Lucene 10.5.	7700
MongoDB	Cơ sở dữ liệu dùng chung, MongoDB 7.	27018

2.2 Các dịch vụ

Nhóm chia thành nhiều dịch vụ nhỏ vì ba lý do. Thứ nhất, mỗi khâu tốn tài nguyên theo một kiểu: tải trang chủ yếu chờ mạng và tốn RAM cho trình duyệt, đọc PDF tốn CPU, nên cần nhân bản riêng từng khâu (hệ chạy bốn bản `downloader`, các khâu khác một bản). Thứ hai, mỗi khâu dùng được ngôn ngữ có thư viện tốt nhất cho nó: Puppeteer trên Node.js, jsoup, Tika và Lucene trên Java. Thứ ba, giữa các khâu chỉ có vài endpoint JSON nên kiểm thử và thay thế từng phần khá dễ.

2.3 Mô hình dữ liệu

Dữ liệu nằm trong database `HUST_2026_IT4863_TEST` gồm bốn collection (Bảng 2.2). Lược đồ được ràng buộc bằng `$jsonSchema` ở mức `strict` (tệp `storage/document/ddl_create_document.js`): `url` phải bắt đầu bằng `http(s)://`, `base64` phải là chuỗi Base64 hợp lệ. Mỗi collection nội dung có chỉ mục duy nhất trên `url`; `graph` có chỉ mục duy nhất trên cặp (`source`, `destinations`) và thêm một chỉ mục phụ trên `destinations`.

Bảng 2.2: Các collection trong MongoDB

Collection	Nội dung
`html`	Trang web: `url`, `format`, `title`, `content` (văn bản đã bóc), `createdAt`, `updatedAt`.
`other`	Tài liệu (pdf, docx, xlsx…): các trường như trên, có thể thêm `origin`.
`picture`	Ảnh: các trường như trên, thêm `base64` chứa byte ảnh.
`graph`	Mỗi bản ghi là một cạnh liên kết: `source`, `destinations` (một url đích), `createdAt`, `updatedAt`.

Chương 3

Thu thập dữ liệu

Chương này đi theo đường đi của một url qua bốn dịch vụ: `mediator` quyết định url nào được tải và khi nào; `downloader` lấy nội dung; `reader` bóc chữ và liên kết; `storage` ghi vào MongoDB.

3.1 Dịch vụ mediator

3.1.1 Vai trò

Mỗi bản `downloader` chỉ tải một url một lúc. Nếu đẩy thẳng hàng nghìn url vào thì nó quá tải, còn nếu gửi dồn vào một website thì website đó có thể chặn. `mediator` đứng giữa để giải quyết cả hai: giữ mọi url trong một hàng đợi bền vững, bỏ url trùng, giãn cách các lần gửi vào cùng một host, chỉ giao url khi có bản `downloader` rảnh và hẹn ngày kiểm tra lại từng url.

`mediator` chạy hai luồng bất đồng bộ trong cùng một tiến trình `asyncio`, gặp nhau ở hàng đợi:

1. **Luồng nhận** (`POST /add`): chuẩn hoá, phân loại và cất url vào hàng đợi, trả lời ngay mà không chờ tải.
2. **Luồng giao**: vòng lặp liên tục, mỗi lượt lấy tối đa một url đến hạn và giao cho một bản `downloader` đang rảnh.

Tách như vậy thì bên gửi url gửi bao nhiêu cũng được trả lời ngay, còn tốc độ giao do tốc độ tải quyết định.

3.1.2 Chuẩn hoá url

Cùng một trang có thể viết bằng nhiều cách. Hàm `canonicalize_url` đưa chúng về một dạng duy nhất để không tải một trang hai lần:

- viết thường scheme và tên miền, bỏ dấu chấm cuối tên miền, bỏ cổng mặc định (80 với http, 443 với https);
- chuẩn hoá đường dẫn (bỏ `.` và `..`), giữ dấu `/` cuối nếu có;

- bỏ phần `#fragment`;
- bỏ các tham số theo dõi và phiên: `utm_*`, `fbclid`, `gclid`, `sessionid`, `jsessionid`, `phpsessid`;
- sắp xếp các tham số còn lại theo khoá rồi theo giá trị.

Ví dụ, `HTTPS://Hust.edu.vn:443/a/../b?z=2&a=1&utm_source=fb#top` trở thành `https://hust.edu.vn/b?a=1&z=2`. Url bị loại nếu không phải `http`/`https`, không có host, có thông tin đăng nhập (`user:pass@`) hoặc có hơn 50 cặp tham số (thường là bẫy crawler sinh url vô hạn).

3.1.3 Phân loại url và chu kỳ crawl lại

Mỗi url thuộc một trong ba loại vì chúng thay đổi với tốc độ khác nhau (Bảng 3.1). Các chu kỳ chỉnh được trong tệp cấu hình YAML; giá trị 0 nghĩa là không crawl lại.

Bảng 3.1: Phân loại url và chu kỳ kiểm tra lại

Loại	Cách nhận biết	Kiểm tra lại sau
`hub` (trang chủ)	Đường dẫn rỗng hoặc `/`; tin mới đăng liên tục.	1 ngày
`page` (trang thường)	Các url còn lại.	7 ngày
`document` (tài liệu, ảnh)	Đuôi `.pdf`, `.doc(x)`, `.xls(x)`, `.ppt(x)`, `.odt`, `.zip`, `.jpg`, `.png`…; hiếm khi đổi.	30 ngày

3.1.4 Hàng đợi bền vững trên SQLite

Hàng đợi là một tệp SQLite có hai bảng. Bảng `urls` lưu mọi url đã biết cùng loại, hạn giao kế tiếp (`next_due_at`), lần gửi cuối và số lần lỗi liên tiếp. Bảng `hosts` lưu thời điểm gửi gần nhất vào từng host.

Listing 3.1: Lược đồ hàng đợi

```
CREATE TABLE urls (
    url TEXT PRIMARY KEY, host TEXT NOT NULL, kind TEXT NOT NULL,
    next_due_at REAL, last_sent_at REAL, attempts INTEGER NOT NULL DEFAULT 0);
CREATE INDEX urls_next_due ON urls (next_due_at);
CREATE TABLE hosts (host TEXT PRIMARY KEY, last_sent_at REAL NOT NULL);
```

Thêm url dùng `INSERT OR IGNORE`: url đã có thì giữ nguyên lịch cũ, nên gửi lại url cũ không làm nó bị tải thêm. Mọi thao tác ghi đều commit ngay, vì vậy tắt dịch vụ rồi bật lại thì việc giao url tiếp tục đúng chỗ đã dừng. Việc chọn url kế tiếp là một truy vấn duy nhất, xét cả điều kiện đến hạn lẫn khoảng nghỉ của host:

Listing 3.2: Chọn url kế tiếp

```
SELECT u.* FROM urls u LEFT JOIN hosts h ON h.host = u.host
WHERE u.next_due_at <= :now
  AND (h.last_sent_at IS NULL OR h.last_sent_at + :host_delay <= :now)
ORDER BY u.next_due_at <= 0 DESC,
         u.kind = 'document' DESC,
         u.next_due_at
LIMIT 1;
```

Thứ tự ưu tiên trong `ORDER BY` là: url được đánh dấu ưu tiên (`next_due_at = 0`), rồi tài liệu và ảnh, rồi url đến hạn sớm nhất. Khoảng nghỉ nằm ngay trong mệnh đề `WHERE`: host chưa nghỉ đủ `host_delay_seconds` (cấu hình 2 giây) bị bỏ qua và truy vấn lấy url của host khác. Nhờ vậy khi một host đang nghỉ, các host khác vẫn được thu thập xen kẽ, và không host nào nhận quá một yêu cầu mỗi hai giây từ toàn hệ thống. Tài liệu được ưu tiên trước trang thường vì đó là phần thông tin có giá trị nhất (quy chế, thông báo) và tải bằng `fetch` nhanh hơn dựng trang bằng trình duyệt.

3.1.5 Luồng giao url

`mediator` làm việc với `downloader` theo hai bước: hỏi `GET /ping` xem đã rảnh chưa (trả `{"status":"đang chờ"}` hoặc `"đang bận"`), nếu rảnh thì gửi `POST /load {"url": ...}`. Các bản `downloader` được duyệt theo vòng (round robin), bắt đầu từ bản sau bản vừa dùng và bỏ qua bản đang bị cho nghỉ vì lỗi trước đó. Thuật toán 1 mô tả một lượt giao.

Một số chi tiết trong cách xử lý:

- **Chuẩn hoá Unicode của trạng thái.** Chuỗi "đang chờ" có thể đến ở dạng NFD (chữ gốc cộng dấu rời) nếu `downloader` được viết trên hệ điều hành khác, nên `mediator` đưa về NFC trước khi so sánh.
- **Cho node lỗi nghỉ.** Node không trả lời `/ping` trong 2 giây, hoặc lỗi khi `/load`, bị bỏ qua 30 giây để không gọi liên tục vào một dịch vụ đang hỏng. Node sống lại thì việc giao tự tiếp tục.
- **Phân biệt lỗi của url và lỗi của node.** Mã 4xx nghĩa là node vẫn khoẻ nhưng từ chối url đó, nên node không bị cho nghỉ còn url được lùi một chu kỳ thay vì thử lại ngay.
- **Giới hạn số lần thử.** Url lỗi 5 lần liên tiếp bị lùi một chu kỳ, nên url hỏng được thử vài lần chứ không lặp mãi.
- **Ưu tiên.** `POST /add` kèm `"priority": true` đặt `next_due_at = 0` cho các url gửi lên (kể cả url đã biết) để chúng được tải ngay. Chức năng "Thêm url" trên giao diện dùng cơ chế này.

Thuật toán 1 Một lượt giao url của mediator

```
u ← url đến hạn kế tiếp (truy vấn ở trên)
if u = ∅ then
    return FALSE                                   ▷ nghỉ 0,5 giây rồi thử lại
end if
for mỗi node n theo thứ tự vòng, bỏ node đang nghỉ do
    s ← PING(n)                                    ▷ hết hạn 2 giây; chuẩn hoá NFC
    if không có phản hồi then
        cho n nghỉ 30 giây; continue
    else if s ≠ "đang chờ" then
        continue
    end if
    r ← LOAD(n, u)                                 ▷ hết hạn 10 giây
    if r = 409 then
        continue                                   ▷ node vừa nhận việc khác
    else if r = 2xx then
        u.next ← now + recrawl(u.kind); ghi thời điểm gửi của host
        return TRUE
    else if r = 4xx then                           ▷ lỗi do url, node vẫn khoẻ
        u.next ← now + recrawl(u.kind); return FALSE
    else                                           ▷ 5xx hoặc hết giờ
        cho n nghỉ 30 giây; u.attempts ← u.attempts + 1
        if u.attempts ≥ 5 then
            u.next ← now + recrawl(u.kind); u.attempts ← 0
        else
            u.next ← now                           ▷ thử lại ở lượt sau
        end if
        return FALSE
    end if
end for
return FALSE
```

3.1.6 Duy trì dữ liệu và tải bài mới

Chu kỳ crawl lại ở Bảng 3.1 chỉ cho biết khi nào tải lại một url đã biết. Còn bài viết mới thì chưa có url nào trong hàng đợi để mà tải lại, nên cần hai cơ chế bổ sung để phát hiện chúng.

Đọc sitemap. Định kỳ `mediator` lấy tệp `sitemap.xml` của từng tên miền con. Url chưa có trong SQLite được thêm vào hàng đợi để tải ngay. Url đã có nhưng `<lastmod>` mới hơn lần tải trước thì bị đặt lại `next_due_at = 0` để tải lại sớm. Không phải website nào cũng có sitemap đầy đủ, nên cơ chế này không thay thế được cơ chế thứ hai bên dưới.

Duyệt trang danh mục theo phân trang. Website khoa, viện thường xếp tin tức thành danh mục nhiều trang (`/tin-tuc?page=2`). Khi có bài mới, chúng xuất hiện ở trang 1 và đẩy bài cũ xuống các trang sau. Hằng ngày `mediator` tải lại trang 1 (loại `hub`,

Bảng 3.1). Nếu toàn bộ bài trên trang 1 đều mới thì có thể còn bài mới ở trang 2, nên cần tải tiếp; nếu đã gặp bài cũ thì dừng. Thuật toán 2 mô tả cách này.

Thuật toán 2 Tải bài mới theo trang danh mục

Vào: trang danh mục đầu tiên u_1; số trang tối đa $K_{\max} = 10$
Ra: mọi bài mới được đưa vào hàng đợi; dừng khi gặp bài đã biết

$k \leftarrow 1$
while $k \leq K_{\max}$ **do**
    tải và bóc trang u_k; $L_k \leftarrow$ các url bài viết trong khối nội dung chính
    $m \leftarrow$ số url trong L_k chưa có trong bảng `urls`
    đưa các url mới đó vào hàng đợi để tải ngay
    if $m < |L_k|$ **then**
        break ▷ đã gặp bài cũ: các bài mới nằm hết ở các trang đã duyệt
    end if
    $u_{k+1} \leftarrow$ url của trang kế tiếp (tăng tham số trang lên $k + 1$)
    đặt u_{k+1} ở mức ưu tiên; $k \leftarrow k + 1$
end while

Có ba trường hợp tại trang k. Nếu $m = 0$, không có bài mới và việc duyệt dừng ngay ở trang 1. Nếu $0 < m < |L_k|$, trang chứa cả bài mới lẫn bài cũ, nghĩa là các bài mới đã nằm trọn trong những trang vừa duyệt nên dừng. Chỉ khi $m = |L_k|$, tức cả trang đều mới, mới có khả năng còn bài mới ở trang sau và thuật toán sang trang $k + 1$. Giới hạn $K_{\max}$ chặn trường hợp một website đăng cả loạt bài cùng lúc, hoặc đổi cách đánh số trang khiến mọi trang đều được coi là mới. Nhờ điều kiện dừng này, mỗi lần cập nhật thường chỉ tải một trang danh mục thay vì toàn bộ chuỗi.

3.1.7 API và cấu trúc mã

`mediator` có bốn endpoint: `POST /add` (nhận một gói url, trả số url mới, trùng, bị loại và url đã chuẩn hoá của từng url), `POST /stop` và `POST /start` (tạm dừng hoặc chạy tiếp việc giao, hàng đợi giữ nguyên), `GET /status` (trạng thái, tổng số url, số url đến hạn). Cứ 30 giây dịch vụ in một dòng tóm tắt vào log.

Listing 3.3: Ví dụ gọi POST /add

```
$ curl -X POST localhost:5200/add -d '{"urls": ["https://hust.edu.vn/",
      "https://HUST.edu.vn:443/#top", "mailto:a@hust.edu.vn"]}'
{"accepted": 1, "duplicates": 1, "rejected": 1,
 "urls": ["https://hust.edu.vn/", "https://hust.edu.vn/", null]}
```

Mã nguồn chia lớp theo kiểu ports-and-adapters:

- `domain/`: logic thuần, không I/O: chuẩn hoá url, phân loại, lập lịch lại, xoay vòng node.
- `application/`: use case `add_urls` và `Dispatcher`, chỉ phụ thuộc vào các giao diện (port) `UrlQueue`, `Downloader`, `Clock`.
- `infrastructure/`: cài đặt các port bằng SQLite và `aiohttp`; đọc cấu hình YAML.

- `api/`: HTTP server `aiohttp`.

Chia như vậy cho phép kiểm thử luồng giao bằng đồng hồ giả và `downloader` giả, không cần mạng hay chờ thời gian thật. Một bài kiểm thử kiến trúc (`test_architecture.py`) bảo đảm lớp `domain` không import lớp ngoài.

3.2 Dịch vụ downloader

`downloader` là một HTTP server Node.js. Nó nhận url qua `POST /load`, trả `202` ngay rồi tải ở nền, xong thì gửi kết quả thẳng sang `reader` (`POST /parse`). Mỗi bản chỉ tải một url một lúc: khi đang tải, `/ping` trả "đang bận" và `/load` trả `409`. Hệ thống chạy bốn bản (`deploy.replicas: 4`), địa chỉ của chúng khai báo trong cấu hình của `mediator`.

3.2.1 Tải trang web bằng trình duyệt headless

Nhiều website của trường nạp nội dung bằng JavaScript nên tải HTML thô là không đủ. `downloader` dùng Puppeteer điều khiển một tiến trình Chromium dùng chung, mỗi lần tải mở một tab:

1. Bật chặn request, huỷ mọi yêu cầu loại `image`, `media`, `font`, `stylesheet`. `reader` chỉ cần DOM, nên việc này vừa nhanh hơn vừa bớt request vào website.
2. Gọi `page.goto(url, {waitUntil: 'load'})` với hạn 30 giây, sau đó chờ mạng rảnh 300 ms (thêm tối đa 2 giây) để các lời gọi AJAX nạp xong nội dung.
3. Nếu mã HTTP ≥ 400 thì ghi log rồi bỏ. Nếu `Content-Type` không phải `text/html` (ví dụ url không có đuôi nhưng trả về PDF), hoặc Chromium huỷ điều hướng (`ERR_ABORTED`) vì đây là tệp tải xuống, thì chuyển sang tải nhị phân.
4. Gửi `page.content()` (DOM sau khi chạy JavaScript) cùng `final_url` sau chuyển hướng sang `reader`.

Khi Chromium chết hoặc mất kết nối, cache của trình duyệt bị xoá để lần tải sau khởi động lại sạch.

3.2.2 Tải tài liệu và ảnh

Url có đuôi tài liệu hoặc ảnh (`pdf`, `docx`, `xlsx`, `pptx`, `jpg`, `png`…) được tải thẳng bằng `fetch`, theo chuyển hướng, giới hạn 50 MB (kiểm tra cả `Content-Length` lẫn số byte thực nhận). Nếu có `Content-Type`, byte của tệp được gửi nguyên trong body `/parse?url=...&final_url=...`, tránh việc Base64 làm dữ liệu to thêm 33%; nếu không thì gửi gói JSON với `encoding: "base64"`.

3.3 Dịch vụ reader

`reader` nhận trang hoặc tệp, bóc chữ và liên kết rồi chuyển tiếp: bản ghi sang `storage`, url `hust.edu.vn` mới sang `mediator`. Dịch vụ viết bằng Java 21, dùng HTTP server có

sẵn của JDK, jsoup để phân tích HTML và Apache Tika 4 để đọc tệp.

3.3.1 Lấy tiêu đề

Tiêu đề lấy theo thứ tự ưu tiên: `<meta property="og:title">`, thẻ `<h1>` đầu tiên, rồi `<title>`. Thẻ `<title>` thường kèm tên website ("Học phí – Trường CNTT&TT") nên để sau cùng. Nếu không có tiêu đề nào thì lấy tên tệp trong url.

3.3.2 Tìm khối nội dung chính

Trang của trường có nhiều phần lặp lại như menu, thanh bên, chân trang. Nếu index cả trang thì trang nào cũng khớp với các từ trong menu. `reader` chọn *khối nội dung chính* bằng một heuristic chấm điểm theo mật độ chữ, cùng ý tưởng với các thuật toán kiểu Readability:

1. Xoá các phần không hiển thị: `script`, `style`, `noscript`, `template`, `iframe` và phần tử có thuộc tính `hidden`.
2. Với mỗi phần tử e có chữ trực tiếp (`ownText`, không tính chữ trong thẻ `<a>`) dài $n \geq 25$ ký tự: leo lên thẻ khối gần nhất e' (một `<span>` nằm trong `<p>` thì tính từ `<p>`), rồi cộng điểm
$$score(\text{cha}(e')) \mathrel{+}= n, \qquad score(\text{ông}(e')) \mathrel{+}= n/2.$$
3. Khối nội dung là phần tử có điểm cao nhất; nếu không có đoạn chữ nào thì lấy `<body>`.

Đoạn chữ ngắn hơn 25 ký tự (mục menu, nhãn nút) không được tính, và chữ nằm trong liên kết cũng vậy, nên một menu toàn liên kết không thể thắng phần thân bài viết. Cộng thêm một nửa điểm cho phần tử ông giúp chọn được vùng bao ngoài khi bài viết bị chia thành nhiều `<div>` con.

3.3.3 Liên kết và văn bản liên kết

`reader` tách liên kết thành hai tập:

- **Liên kết nằm trong khối nội dung** (`<a href>` và `<img src>`) được ghi vào đồ thị `graph`. Văn bản liên kết là *câu chứa liên kết* chứ không chỉ chữ trong thẻ: `reader` duyệt các nút chữ của khối câu gần nhất (`p`, `li`, `td`, `h1`–`h6`…) và cắt từ dấu kết câu gần nhất bên trái đến dấu kết câu đầu tiên bên phải. Nhờ vậy liên kết "xem tại đây" vẫn mang ngữ cảnh "Quy chế đào tạo 2025, xem tại đây". Với ảnh, văn bản lấy từ `alt`, không có thì lấy `title`.
- **Mọi liên kết của trang**, kể cả menu và chân trang, được lọc lấy các url có host `hust.edu.vn` hoặc `*.hust.edu.vn` rồi gửi về `mediator`. Menu cũng được dùng để mở rộng crawl, vì menu chính là bản đồ của website.

Liên kết được đổi thành url tuyệt đối theo `final_url`, bỏ fragment, bỏ liên kết về chính trang và bỏ trùng. Ký tự không hợp lệ (dấu cách, chữ có dấu) được mã hoá `%XX` để `storage` không từ chối cả bản ghi.

3.3.4 Đọc tài liệu và ảnh

Tệp nhị phân được đưa qua `AutoDetectParser` của Tika, nó tự nhận dạng định dạng theo nội dung chứ không tin vào đuôi tệp. Kết quả:

- **Tài liệu** (PDF, DOC/DOCX, XLS/XLSX, PPT/PPTX): lấy chữ thuần, cắt ở 1 triệu ký tự. Tiêu đề lấy từ metadata của tệp, không có thì lấy tên tệp. Định dạng ghi theo đuôi tệp, không có đuôi thì suy từ MIME.
- **Ảnh**: không OCR. Hệ thống lưu tiêu đề (tên tệp), metadata đã chuẩn hoá (MIME, kích thước điểm ảnh, ngày chụp, tác giả, máy ảnh) và byte ảnh dạng Base64. Ảnh trên 11 MB bị bỏ qua, vì Base64 làm dữ liệu to thêm một phần ba, cộng với các trường khác sẽ vượt giới hạn 16 MB của một bản ghi MongoDB.

Tệp không đọc được trả `415` hoặc `422`; trang không bóc được chữ trả `422`. Bảng mã của trang lấy từ `charset` trong `Content-Type`, mặc định UTF-8.

3.3.5 Chuyển tiếp

Bản ghi được gửi bằng `POST /add` sang `storage`, danh sách url hust gửi sang `POST /add` của `mediator`. Mỗi lời gọi thử lại một lần với hạn 10 giây; nếu vẫn thất bại thì `/parse` trả `502` kèm tên dịch vụ và lý do. Mỗi request ghi một dòng log gồm url, định dạng, mã trạng thái, thời gian xử lý và số liên kết.

3.4 Dịch vụ storage

`storage` là ứng dụng Spring Boot với một endpoint ghi (`POST /add`) và hai endpoint đọc: `GET /stats` (số bản ghi, dung lượng từng collection, bản ghi mới nhất) và `GET /record` (xem một bản ghi theo url).

Listing 3.4: Body của POST /add trên storage

```
{"url": "https://hust.edu.vn/tin-tuc1", "format": "html", "title": "Tin tức 1",
 "content": "Cơ hội thực tập có lương tại Nhật cho sinh viên HUST.",
 "out-links": [{"url": "https://hust.edu.vn/tin-tuc2", "text": "Thông tin tuyển
    dụng"}]}
```

Các bước xử lý:

1. **Kiểm tra.** `url` và từng `out-links.url` phải là url http(s) tuyệt đối, không có thông tin đăng nhập; `format` khớp `[a-z0-9][a-z0-9.+_-]{0,31}`; `title` không rỗng; phải có `content` hoặc `base64`, và `base64` phải giải mã được. Sai thì trả `400` kèm lý do.
2. **Chọn collection theo `format`.** `html`/`htm` vào `html`; `jpg`, `png`, `gif`, `webp`, `svg`, `bmp` vào `picture`; còn lại vào `other`.
3. **Upsert theo url** bằng `findAndModify(upsert=true)`. Crawl lại một trang thì nội dung được ghi đè, `updatedAt` được cập nhật, `createdAt` giữ nguyên. Hai request cùng

url chạy song song có thể cùng chèn và gặp `DuplicateKeyException`; khi đó thao tác được làm lại ở dạng cập nhật.

4. **Ghi cạnh đồ thị.** Mỗi liên kết ra là một bản ghi (`source`, `destinations`), upsert theo cặp nên không có cạnh trùng.

Mất kết nối MongoDB thì trả `503`, các lỗi ghi khác trả `500`. Mỗi request được gán một `X-Request-ID` và ghi log đầu vào, đầu ra, thời gian (byte Base64 được rút gọn trong log).

3.5 Nạp url mầm và theo dõi

Tệp `link_all.txt` chứa 101 461 url mầm thuộc miền `hust.edu.vn`. Script `feed.py` chờ `mediator` sẵn sàng, gửi danh sách theo lô (mặc định 2 000 url mỗi lô) rồi gọi `/start`. Nạp lại nhiều lần cũng không sao vì url đã có sẽ bị bỏ qua. Sau đó crawl tự mở rộng theo các liên kết mà `reader` tìm được.

Trang `dashboard.html` (do dịch vụ `search` phục vụ tại `http://localhost:7700/`) có ba thẻ: *Theo dõi crawl* (trạng thái `mediator`, số bản ghi từng collection lấy từ `/stats`, các bản ghi mới nhất), *Tìm kiếm* và *Thêm url*. Ở thẻ Thêm url, người dùng dán một hoặc nhiều url; các url được gửi tới `mediator` với `priority: true`, đi qua luồng crawl bình thường, rồi được index ngay bằng `POST /index` mà không cần dựng lại cả chỉ mục.

3.6 Kết quả thu thập

Crawler chạy từ ngày 26/10/2026 đến 31/10/2026. Bảng 3.2 tổng hợp số liệu từ bản export của MongoDB.

Bảng 3.2: Số liệu dữ liệu thu thập được

Collection	Số bản ghi	Ghi chú
`html`	28 168	95 tên miền con; độ dài nội dung trung vị 1 079 ký tự, trung bình 2 575 ký tự
`other`	4 690	25 tên miền con; PDF 4 289, DOCX 164, DOC 155, XLSX 61, XLS 16, PPTX 5; độ dài trung vị 4 795 ký tự
`picture`	9 614	Ảnh kèm metadata và byte Base64 (khoảng 7,5 GB)
`graph`	84 673	44 095 nút; 19 388 trang có liên kết ra; 79,8% cạnh trỏ vào miền `hust.edu.vn`

Số trang giữa các host lệch nhiều (Bảng 3.3): ba host lớn nhất chiếm 52% số trang web. Nguyên nhân là mỗi host được thu thập với cùng một tốc độ tối đa, nên host nào có nhiều trang và liên kết dày (website khoa có nhiều bài tin tức) thì tích lũy được nhiều trang hơn trong cùng thời gian. Hơn một nửa số tài liệu đến từ cổng chính và bản thử nghiệm `hust-dev`, nơi đăng các văn bản quy chế, thông báo. Đồ thị có 44 095 nút nhưng

Bảng 3.3: Các tên miền con có nhiều trang nhất

Host (`html`)	Số trang	Host (`other`)	Số tài liệu
`bktextile.hust.edu.vn`	5 567	`hust-dev.hust.edu.vn`	1 735
`clc.hust.edu.vn`	5 444	`hust.edu.vn`	1 418
`fami.hust.edu.vn`	3 806	`library.hust.edu.vn`	680
`bme.hust.edu.vn`	1 535	`ctt.hust.edu.vn`	278
`icea.hust.edu.vn`	1 485	`fami.hust.edu.vn`	205
`inest.hust.edu.vn`	1 396	`sep.hust.edu.vn`	146
`hust-dev.hust.edu.vn`	1 233	`sdh.hust.edu.vn`	67
`hust.edu.vn`	981	`soict.hust.edu.vn`	53

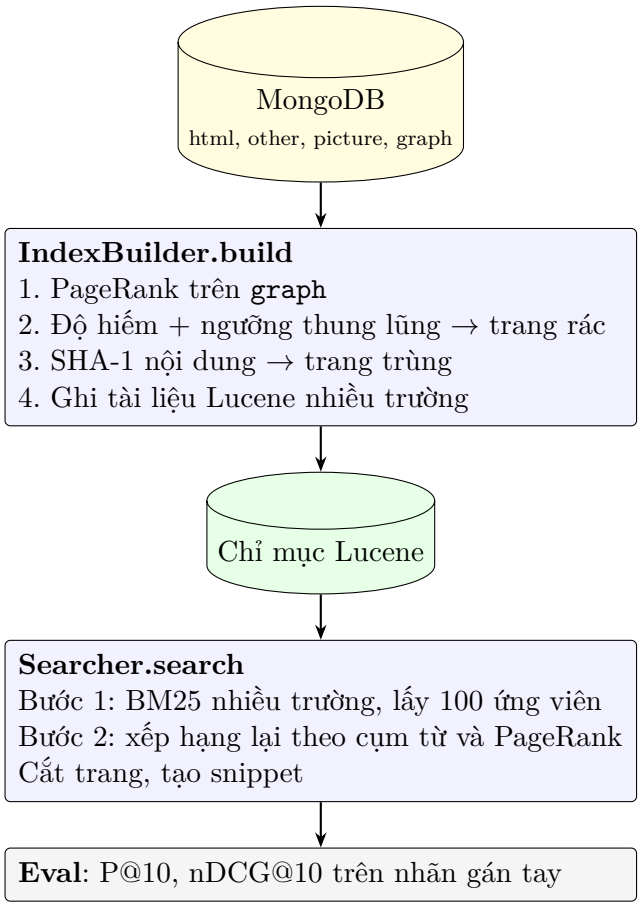
chỉ 19 388 trang có liên kết ra; phần còn lại là url được trỏ tới nhưng chưa crawl xong, hoặc là tệp, ảnh, trang ngoài trường.

Chương 4

Máy tìm kiếm

4.1 Tổng quan

Dịch vụ `search` (Java 21, Lucene 10.5) có hai quy trình (Hình 4.1): *dựng chỉ mục* từ MongoDB (lúc khởi động nếu chưa có chỉ mục, hoặc khi gọi `POST /reindex`) và *trả lời truy vấn* qua `GET /search`.



Hình 4.1: Hai quy trình của dịch vụ search

Nguồn dữ liệu được trừu tượng hoá bằng giao diện `Source` với hai hàm `records()` và `edges()`. `MongoSource` đọc ba collection nội dung (chỉ các trường `url`, `format`, `title`,

`content`, `updatedAt`, không bao giờ lấy `base64`) và collection `graph`. `JsonFileSource` đọc một tệp JSON dạng mảng `{url, title, content, olinks}`; dịch vụ `search-json` (cổng 7701) dùng nguồn này để chạy cùng mã trên một bộ dữ liệu khác. Mỗi bản ghi tương ứng một tài liệu Lucene.

4.2 Phân tích văn bản tiếng Việt

4.2.1 Chuẩn hoá Unicode

Tiếng Việt có hai cách mã hoá cùng một chữ có dấu: ký tự dựng sẵn ("ệ" là U+1EC7) và ký tự gốc cộng dấu rời ("e" + U+0302 + U+0323). Hai chuỗi trông giống hệt nhau nhưng khác byte, nên chữ "ệ" gõ ở máy này có thể không khớp với "ệ" lưu ở máy khác. Vì vậy mọi chuỗi đều được chuẩn hoá về NFC (`Analyzers.nfc`) trước khi index và trước khi truy vấn.

4.2.2 Tách từ: unigram âm tiết cộng bigram

Tiếng Việt viết các âm tiết cách nhau bằng dấu cách, còn một từ có thể gồm nhiều âm tiết ("học phí", "bách khoa"). Nhóm không dùng bộ tách từ riêng (cần thêm mô hình và thư viện) mà dùng `StandardTokenizer` của Lucene, tách theo chuẩn Unicode UAX #29 nên mỗi âm tiết thành một token, rồi thêm bigram âm tiết để bắt các từ ghép. Có bốn analyzer (Bảng 4.1).

Bảng 4.1: Các analyzer, ví dụ với "Bách Khoa Hà Nội"

Analyzer	Chuỗi xử lý	Token
`vi`	`StandardTokenizer` → chữ thường	`bách`, `khoa`, `hà`, `nội`
`viBi`	`vi` → `ShingleFilter(2,2)`, không giữ unigram	`bách khoa`, `khoa hà`, `hà nội`
`viAscii`	`vi` → `ASCIIFoldingFilter` (bỏ dấu, `đ`→`d`)	`bach`, `khoa`, `ha`, `noi`
`urlTokens`	Tách theo mọi ký tự không phải chữ hoặc số → chữ thường → bỏ dấu	`.../hoc-phi-654580.html` → `hoc`, `phi`, `654580`, `html`

Như vậy "học phí" được biểu diễn bằng hai unigram `học`, `phí` và một bigram `học phí`. Tài liệu chứa đúng cụm "học phí" khớp cả ba token nên điểm cao hơn tài liệu chỉ chứa "học" và "phí" ở hai chỗ rời nhau. Bigram cũng sinh ra những cặp không phải từ (`khoa hà`), nhưng các cặp này hiếm khi trùng với truy vấn nên ít gây nhiễu.

4.2.3 Bản bỏ dấu

Người dùng hay gõ không dấu (“lich thi”), và url, tên tệp trên website của trường cũng thường không dấu (`lich-thi-hk1.pdf`). Vì thế mỗi trường văn bản được index thêm một bản bỏ dấu. Bản này có trọng số thấp hơn bản có dấu, để khi truy vấn có dấu thì trang khớp đúng dấu đứng trên trang chỉ khớp sau khi bỏ dấu (“hồ” khác “hổ”, nhưng bỏ dấu đều thành “ho”).

4.3 Lược đồ chỉ mục

Bảng 4.2: Các trường của tài liệu Lucene

Trường	Loại	Analyzer	Lưu	Mục đích
`url`, `host`, `collection`, `format`	String	—	có	Định danh, lọc theo host và loại
`title`	Text	`vi`	có	Khớp tiêu đề
`content`	Text (vị trí, offset)	`vi`	có	Khớp nội dung, cụm từ, snippet
`title_bi`, `content_bi`	Text	`viBi`	không	Khớp cụm hai âm tiết
`title_ascii`, `content_ascii`	Text	`viAscii`	không	Khớp truy vấn không dấu
`url_tokens`	Text	`urlTokens`	không	Khớp từ trong url
`sig:pagerank`	FeatureField	—	không	Tín hiệu liên kết
`updatedAt`	Stored	—	có	Trả về cho client
`junk`	String	—	có	`spam` hoặc `dup`, chỉ có khi bị đánh dấu

Trường `content` được index với `DOCS_AND_FREQS_AND_POSITIONS_AND_OFFSETS`: vị trí cần cho truy vấn cụm từ ở bước xếp hạng lại, offset cần cho `UnifiedHighlighter` tạo snippet mà không phải phân tích lại văn bản.

4.4 BM25 theo trường

Điểm văn bản dùng BM25 [2]. Với truy vấn q và tài liệu d trên trường f:

$$\mathrm{BM25}_f(d,q) = \sum_{t \in q} \mathrm{IDF}(t) \cdot \frac{tf_{t,d}}{tf_{t,d} + k_1 \left(1 - b + b\,\dfrac{|d|_f}{avgdl_f}\right)}, \tag{4.1}$$

$$\mathrm{IDF}(t) = \ln\!\left(1 + \frac{N - n_t + 0{,}5}{n_t + 0{,}5}\right) \tag{4.2}$$

trong đó $tf_{t,d}$ là số lần t xuất hiện trong trường f của d, $|d|_f$ là độ dài trường, $avgdl_f$ là độ dài trung bình của trường, N là số tài liệu và n_t là số tài liệu chứa t. (Lucene bỏ thừa số $k_1 + 1$ ở tử số; điều này không làm đổi thứ hạng.)

Mọi trường dùng $k_1 = 1{,}2$. Tham số chuẩn hoá độ dài b khác nhau theo trường: $b = 0{,}75$ cho tiêu đề và url, $b = 0{,}5$ cho các trường nội dung. Lý do là kho có cả trang web ngắn (trung vị khoảng 1 000 ký tự) lẫn PDF dài (trung bình hơn 26 000 ký tự). Với $b = 0{,}75$, các văn bản quy chế dài bị phạt quá nặng dù chúng thường chính là kết quả cần tìm.

4.5 PageRank

4.5.1 Mô hình

PageRank [3] là xác suất dừng của một người duyệt web ngẫu nhiên: ở mỗi bước, với xác suất $d = 0{,}85$ người đó bấm vào một liên kết bất kỳ trên trang hiện tại, với xác suất $1 - d$ thì nhảy sang một trang bất kỳ.

$$\mathrm{PR}(v) = \frac{1-d}{N} + d\left(\sum_{u \to v} \frac{\mathrm{PR}(u)}{\mathrm{outdeg}(u)} + \frac{D}{N}\right), \qquad D = \sum_{u:\,\mathrm{outdeg}(u)=0} \mathrm{PR}(u) \tag{4.3}$$

Ở đây D là tổng PageRank của các nút cụt (không có liên kết ra), được chia đều cho mọi nút để tổng PageRank luôn bằng 1.

4.5.2 Cài đặt

Đồ thị đọc từ collection `graph`. Khi nạp, cạnh thiếu đầu mút và cạnh tự trỏ bị bỏ; cạnh trùng bị bỏ bằng khoá $(a \ll 32) \mid b$ trên chỉ số nút. Mỗi url là một nút, đánh số theo thứ tự gặp, các cạnh lưu trong hai mảng `int`. Giá trị khởi tạo là $1/N$; thuật toán lặp tối đa 50 vòng và dừng sớm khi $\sum_v |\mathrm{PR}_{k+1}(v) - \mathrm{PR}_k(v)| < 10^{-6}$. Mỗi vòng tốn $O(N + E)$ và chạy hoàn toàn trong bộ nhớ; với đồ thị 44 nghìn nút và 85 nghìn cạnh, thời gian tính nhỏ so với thời gian ghi chỉ mục.

4.5.3 Đưa vào điểm xếp hạng

Lúc index, giá trị lưu vào `FeatureField` là $s = \max(\mathrm{PR} \cdot N,\ 10^{-4})$. Nhân với N để giá trị trung bình bằng 1, không phụ thuộc kích thước đồ thị; tài liệu không có trong đồ thị nhận giá trị sàn 10^{-4}. Lúc xếp hạng lại, PageRank đóng góp qua hàm bão hoà

$$\mathrm{sat}(s) = w \cdot \frac{s}{s + \mathit{pivot}}, \qquad w = 2, \tag{4.4}$$

trong đó `pivot` do Lucene ước lượng từ phân bố giá trị trong chỉ mục. Hàm này tăng nhanh ở giá trị thấp rồi phẳng dần, và bị chặn trên bởi w. Nhờ vậy trang chủ có PageRank gấp hàng nghìn lần trung bình cũng không lấn át hoàn toàn điểm văn bản.

4.6 Phát hiện trang rác bằng độ hiếm

4.6.1 Quan sát

Một số trang html trên website của trường bị chèn nội dung rác (từ khoá quảng cáo, văn bản lạ không liên quan tới trường). Vốn từ của những trang này gần như không trùng với phần còn lại của kho, trong khi trang thật dùng từ vựng chung của trường (“đào tạo”, “sinh viên”, “thông báo”). Từ đó nhóm định nghĩa *độ hiếm* của một trang là IDF trung bình của các từ trong trang.

4.6.2 Độ đo

Với trang html p có ít nhất 50 từ, gọi W_p là dãy các từ (chuỗi chữ cái `\p{L}+`, chữ thường, NFC) của tiêu đề và nội dung, N là số trang html đủ dài, và $\mathit{df}(w)$ là số trang html đủ dài chứa w:

$$\mathrm{odd}(p) = \frac{1}{|W_p|} \sum_{w \in W_p} \ln \frac{N}{\mathit{df}(w)}. \tag{4.5}$$

Từ phổ biến cho giá trị gần 0, từ chỉ xuất hiện ở một trang cho giá trị $\ln N$. Trang dưới 50 từ không đủ để ước lượng nên không bao giờ bị coi là rác; tài liệu và ảnh không xét.

4.6.3 Chọn ngưỡng tự động

Ngưỡng phụ thuộc vào kho dữ liệu nên nhóm không đặt bằng tay. Histogram độ hiếm của kho có dạng hai cụm: cụm lớn bên trái là trang thật, cụm nhỏ nằm cao hơn là trang rác. Ngưỡng được chọn ở đáy thung lũng giữa hai cụm, theo phương pháp của Prewitt và Mendelsohn [4] (Thuật toán 3).

Đáy thung lũng được đo trên histogram gốc vì làm mượt nhiều vòng sẽ lấp mất thung lũng. Điều kiện $\mathit{raw}[v] \le 0{,}5\, h[b]$ loại các thung lũng nông do nhiễu. Trang có độ hiếm vượt ngưỡng được gắn `junk:spam`. Ngưỡng của lần dựng gần nhất được ghi vào log và trả ở `GET /ping` (`stats.oddityThreshold`). Cách làm này có giới hạn: nếu số trang rác

Thuật toán 3 Chọn ngưỡng độ hiếm theo thung lũng của histogram

Vào: điểm độ hiếm $o_1, \dots, o_m$; độ rộng bin $\beta = 0{,}1$
Ra: ngưỡng θ ($+\infty$ nếu không có cụm rác)

$h \leftarrow$ histogram của o với bin rộng β bắt đầu từ $\min o$; $raw \leftarrow h$
for $r = 1$ **to** 200 **do**
  $P \leftarrow$ các đỉnh của h ▷ cao hơn bên trái, không thấp hơn bên phải; dải bằng nhau tính một đỉnh
  if $|P| < 2$ **then return** $+\infty$
  end if
  if $|P| = 2$, gọi hai đỉnh là $a < b$ **then**
    if $h[b] > h[a]$ **then return** $+\infty$ ▷ cụm bên phải đông hơn thì không phải rác
    end if
    $v \leftarrow \arg\min_{a \le i \le b} raw[i]$ ▷ đáy đo trên histogram gốc
    if $raw[v] \le 0{,}5 \cdot h[b]$ **then return** $\min o + (v + 0{,}5)\beta$
    end if
  end if
  $h[i] \leftarrow (h[i-1] + h[i] + h[i+1])/3$ với mọi i ▷ làm mượt
end for
return $+\infty$

quá ít, không đủ tạo thành một đỉnh riêng, thì chúng không bị phát hiện. Khi đó có thể đổi sang ngưỡng cố định theo phân vị.

4.7 Loại trang trùng nội dung

Trang mirror, bản in, bản tiếng Việt và tiếng Anh giống hệt nhau, hay cùng một PDF đăng ở nhiều nơi đều làm trang kết quả bị lặp. Mỗi tài liệu (cả ba collection) có nội dung từ 200 ký tự trở lên được gán một khoá

$$\text{key}(p) = \text{SHA1}\big(\text{trim}(\text{collapse_ws}(\text{lower}(\text{NFC}(\textit{content}))))\big). \tag{4.6}$$

Với mỗi khoá có nhiều url, chỉ giữ *một* url: url có PageRank cao nhất, nếu bằng nhau thì lấy url ngắn hơn; trang đã bị đánh dấu rác không được làm bản giữ lại. Các url còn lại của nhóm bị gắn `junk:dup`. Cách này chỉ bắt được trùng khớp hoàn toàn sau chuẩn hoá, không bắt được trang gần trùng.

4.8 Quy trình dựng chỉ mục

`IndexBuilder.build` dựng lại toàn bộ chỉ mục (`OpenMode.CREATE`). Để không phải giữ nội dung cả kho trong bộ nhớ, nó duyệt nguồn dữ liệu nhiều lượt và giữa các lượt chỉ giữ url cùng các con số:

1. Đếm cạnh và tính PageRank.
2. Lượt 1, trên các trang html đủ dài: đếm *df* của từng từ.

3. Lượt 2: tính độ hiếm của từng trang html và khoá SHA-1 của mọi tài liệu; chọn ngưỡng rác; chọn bản giữ lại cho mỗi nhóm trùng.
4. Lượt 3: ghi từng tài liệu Lucene kèm cờ `junk`.

`POST /reindex` chạy việc dựng ở luồng nền (trả `202`, đang dựng thì trả `409`) rồi gọi `SearcherManager.maybeRefresh()`, nên tìm kiếm vẫn phục vụ được trong lúc dựng. `POST /index?url=...` thêm hoặc thay một trang đã có trong MongoDB vào chỉ mục hiện tại bằng `updateDocument`; trang thêm lẻ chưa có PageRank và chưa được kiểm tra rác cho đến lần dựng lại đầy đủ kế tiếp. Sau mỗi lần dựng, log ghi số tài liệu, số trang rác và ngưỡng, số trang trùng, số cạnh và tỉ lệ tài liệu có mặt trong đồ thị liên kết.

4.9 Truy vấn hai bước

$q \to$ NFC, trim $\to$ truy vấn BM25 nhiều trường $\to$ 100 ứng viên $\to$ xếp hạng lại (profile C, D) $\to$ cắt trang $\to$ snippet

Việc chia hai bước tách phần rẻ khỏi phần đắt: BM25 trên chỉ mục ngược tính được cho mọi tài liệu khớp, còn truy vấn cụm từ (phải đọc vị trí) và các tín hiệu ngoài văn bản chỉ cần tính cho 100 ứng viên tốt nhất.

4.9.1 Bước 1: lấy ứng viên bằng BM25 nhiều trường

Với mỗi trường, `QueryBuilder` phân tích truy vấn bằng analyzer của trường đó rồi nối các token bằng OR (tài liệu chỉ cần khớp một token). Điểm bước 1 là tổng có trọng số:

$$S_1(d, q) = \sum_{f \in F} w_f \cdot \mathrm{BM25}_f(d, q). \tag{4.7}$$

Bảng 4.3: Trọng số trường ở bước lấy ứng viên

Trường	Truy vấn có dấu	Truy vấn không dấu (từ B)	Có từ profile
`url_tokens`	1,5	1,5	A
`title`	3	—	A
`content`	1	—	A
`title_bi`	3	—	B
`content_bi`	1,5	—	B
`title_ascii`	2	3	B
`content_ascii`	0,7	1	B

Các trọng số ở Bảng 4.3 được chọn như sau. Tiêu đề do người viết chọn để tóm tắt bài nên nặng gấp ba nội dung. Bigram trong nội dung nặng hơn unigram vì khớp cả cụm

hai âm tiết là bằng chứng mạnh hơn khớp rời. Bản bỏ dấu nhẹ hơn bản có dấu để trang đúng dấu đứng trên.

Truy vấn không dấu. Từ profile B, nếu truy vấn không có dấu nào (tức bằng chính nó sau khi bỏ dấu, ví dụ "lich thi") thì chỉ hỏi các trường bỏ dấu (`title_ascii` trọng số 3, `content_ascii` trọng số 1) cộng với `url_tokens`. Nếu vẫn hỏi cả các trường có dấu, trang chỉ có tên tệp không dấu sẽ được ưu ái hơn trang "Lịch thi" viết đúng dấu, trong khi người dùng gõ không dấu chỉ vì tiện chứ không phải vì muốn tìm chữ không dấu.

Bộ lọc. Tham số `type` lọc theo `collection`, `host` lọc theo host; cả hai là mệnh đề `FILTER` nên không tham gia chấm điểm. Profile D thêm hai mệnh đề `MUST_NOT` để loại tài liệu có `junk:spam` hoặc `junk:dup`.

4.9.2 Bước 2: xếp hạng lại

`QueryRescorer` xếp hạng lại 100 tài liệu đầu bằng một truy vấn phụ là OR của ba mệnh đề:

$$S_2(d,q) = S_1(d,q) + 2 \cdot \text{Phrase}_{\texttt{title}}(d,q) + 1 \cdot \text{Phrase}_{\texttt{content}}(d,q) + 2 \cdot \frac{s_d}{s_d + pivot}, \quad (4.8)$$

trong đó Phrase_f là điểm truy vấn cụm từ trên trường f với độ lệch (slop) bằng 2: các token của truy vấn phải xuất hiện gần nhau, đúng thứ tự, cho phép chèn tối đa hai vị trí. Mệnh đề cụm từ bị bỏ khi truy vấn chỉ có một token. Thành phần cuối là PageRank bão hoà.

4.9.3 Bốn profile xếp hạng

Các kỹ thuật được bật dần qua bốn profile (Bảng 4.4), để đo đóng góp của từng tầng bằng đánh giá offline trên cùng một chỉ mục. Giao diện mặc định dùng C; khi đánh dấu "Lọc rác" thì dùng D.

Bảng 4.4: Bốn profile xếp hạng

Profile	Lấy ứng viên	Xếp hạng lại	Lọc rác, trùng
A	BM25 trên `url_tokens`, `title`, `content`	không	không
B	A + bigram + bỏ dấu, nhánh truy vấn không dấu	không	không
C (mặc định)	như B	cụm từ + PageRank	không
D	như B	như C	có

4.9.4 Snippet

Snippet lấy tối đa hai đoạn của `content` bằng `UnifiedHighlighter`, bọc từ khớp trong `<b>…</b>` và nối các đoạn bằng dấu "…". Với truy vấn không dấu chỉ khớp `content_ascii`, highlight chạy trên trường bỏ dấu: `ASCIIFoldingFilter` giữ nguyên offset nên vị trí trong bản bỏ dấu trùng với vị trí trong văn bản gốc, và dùng lại được để tô đậm trên nội dung có dấu. Nếu vẫn không có đoạn nào thì lấy 200 ký tự đầu, đã escape HTML.

4.9.5 Ví dụ

Truy vấn "học phí" với profile D:

1. Lấy ứng viên bằng truy vấn Lucene dưới đây, loại tài liệu rác và trùng, giữ 100 tài liệu đầu:

```
url_tokens:(học phí)^1.5  title:(học phí)^3      content:(học phí)
title_bi:"học phí"^3      content_bi:"học phí"^1.5
title_ascii:(hoc phi)^2   content_ascii:(hoc phi)^0.7
-junk:spam  -junk:dup
```

2. Xếp hạng lại: trang có tiêu đề chứa đúng cụm "học phí" được cộng điểm cụm từ nhân 2; trang được nhiều trang quan trọng trỏ tới được cộng điểm PageRank.
3. Trả 10 kết quả đầu kèm snippet.

4.10 API tìm kiếm

Bảng 4.5: Các endpoint của dịch vụ search

Endpoint	Chức năng
`GET /search`	Tìm kiếm. Tham số: `q`; `type` (`html`, `other`, `picture`); `host`; `profile` (A–D, mặc định C); `page`, `size` (≤ 50, $page \times size \leq 100$).
`GET /ping`	Số tài liệu, lần index cuối, thống kê (rác, trùng, ngưỡng độ hiếm, số cạnh).
`POST /reindex`	Dựng lại chỉ mục ở nền.
`POST /index?url=`	Index ngay một trang đã có trong MongoDB (404 nếu chưa có).
`GET /`	Giao diện `dashboard.html`.

Listing 4.1: Ví dụ truy vấn và kết quả

```
$ curl -G localhost:7700/search --data-urlencode 'q=học phí' -d profile=D -d size=10
{"q": "học phí", "profile": "D", "total": 412, "tookMs": 18, "page": 1,
 "hits": [{"url": "https://hust.edu.vn/vi/sinh-vien/hoc-bong-hoc-phi/hoc-phi-654580.
    html",
```

```
          "title": "HỌC PHÍ", "collection": "html", "format": "html",
          "host": "hust.edu.vn", "score": 9.31,
          "snippet": "... mức <b>học</b> <b>phí</b> ...", "updatedAt":
   1761200000000}, ...]}
```


Trường `total` là số tài liệu khớp thật (`IndexSearcher.count`), không chỉ 100. Tuy vậy chỉ 100 kết quả đầu được xếp hạng nên phân trang dừng ở đó.

Chương 5

Đánh giá

5.1 Mục tiêu và dữ liệu

Đánh giá offline so sánh bốn profile A–D trên cùng một tập truy vấn để xem bigram và bỏ dấu, xếp hạng lại, lọc rác có thật sự cải thiện thứ hạng hay không. Công cụ là lớp `Eval`, chạy trên cùng thư mục chỉ mục với dịch vụ.

Tập truy vấn `search/eval/queries.tsv` gồm 25 truy vấn mô phỏng nhu cầu của sinh viên và người ngoài trường, trộn nhiều dạng:

- có dấu, nhiều âm tiết: "học bổng khuyến khích học tập", "chuẩn đầu ra tiếng anh", "bảo hiểm y tế sinh viên";
- không dấu: "lich thi", "thong bao nghi tet", "thu vien sach dien tu";
- tên đơn vị: "soict tuyển sinh", "viện điện", "trường cơ khí", "dệt may";
- có năm hoặc tiếng Anh: "học phí 2025", "hust admission".

Tệp nhãn `qrels.tsv` có các cột `qid`, `grade`, `url`, `title`, với `grade` $\in \{0, 1, 2\}$ lần lượt là không liên quan, liên quan một phần và liên quan hoàn toàn.

5.2 Quy trình

1. **Pooling.** Chạy cả bốn profile cho mỗi truy vấn rồi gộp top 10 của từng profile vào `qrels.tsv` (`Eval --pool`). Nhãn cũ được giữ, chỉ thêm các cặp (`qid`, `url`) mới, nên chạy lại sau khi sửa thuật toán không mất công gán nhãn. Pooling tránh thiên vị về một hệ: tài liệu nào được bất kỳ profile nào xếp cao đều được người gán nhãn xem.
2. **Gán nhãn tay** vào cột `grade`.
3. **Chấm điểm** bằng `Eval` (thêm `--detail` để in điểm từng truy vấn).

5.3 Độ đo

Mỗi profile chạy từng truy vấn không lọc `type`/`host` và lấy top $K = 10$. Kết quả chưa có nhãn được tính là grade 0 và được đếm riêng, để biết số liệu còn thiếu nhãn đến đâu.

Precision@10. Tỉ lệ kết quả liên quan trong top 10, không xét thứ tự:

$$\text{P@10} = \frac{|\{i \le 10 : g_i \ge 1\}|}{10}. \tag{5.1}$$

nDCG@10 [5]. Độ lợi là $2^g - 1$ (grade 2 có độ lợi 3, gấp ba grade 1), chiết khấu theo vị trí:

$$\text{DCG@10} = \sum_{i=1}^{10} \frac{2^{g_i} - 1}{\log_2(i+1)}, \qquad \text{nDCG@10} = \frac{\text{DCG@10}}{\text{IDCG@10}}, \tag{5.2}$$

trong đó IDCG là DCG của thứ hạng lý tưởng, lấy từ *mọi* nhãn đã gán của truy vấn (sắp giảm dần, cắt ở 10). Truy vấn chưa có nhãn nào có IDCG $= 0$ và nDCG $= 0$, nhưng vẫn được tính vào trung bình. Điểm của một profile là trung bình trên các truy vấn.

5.4 Kết quả

Bước pooling đã sinh 424 cặp (truy vấn, tài liệu) ứng viên trong `qrels.tsv`, trung bình khoảng 17 tài liệu khác nhau cho mỗi truy vấn (tối đa 40 nếu bốn profile không có kết quả nào trùng nhau). Con số này cho thấy top 10 của các profile khác nhau khá nhiều, tức là các kỹ thuật bổ sung thật sự làm thay đổi thứ hạng.

Tại thời điểm viết báo cáo, cột `grade` chưa được gán nhãn đầy đủ nên nhóm chưa công bố số liệu P@10 và nDCG@10. Bảng 5.1 là khuôn kết quả do công cụ `Eval` in ra; chỉ nên tin các con số khi cột "chưa gán nhãn" gần bằng 0.

Bảng 5.1: Kết quả đánh giá offline trên 25 truy vấn (sẽ điền sau khi gán nhãn)

Profile	P@10	nDCG@10	Kết quả chưa gán nhãn
A	–	–	–
B	–	–	–
C	–	–	–
D	–	–	–

Cách đọc kết quả khi đã có số liệu:

- **B so với A** cho biết đóng góp của bigram và bỏ dấu. Kỳ vọng rõ nhất ở các truy vấn không dấu ("lich thi", "thong bao nghi tet"), vì profile A không có nhánh bỏ dấu nên không khớp được với trang có dấu.
- **C so với B** cho biết đóng góp của cụm từ và PageRank. Kỳ vọng ở các truy vấn nhiều âm tiết ("quy chế đào tạo") và truy vấn tìm trang chính của một đơn vị ("viện điện", "soict tuyển sinh").
- **D so với C** cho biết đóng góp của lọc rác và loại trùng. Kỳ vọng ở các truy vấn mà kết quả bị lặp (cùng một thông báo đăng ở nhiều website khoa) hoặc lẫn trang rác.

Chương 6

Triển khai và kiểm thử

6.1 Triển khai

Toàn bộ hệ thống được mô tả trong một tệp `compose.yaml`. Mỗi dịch vụ có `Dockerfile` riêng; dữ liệu bền vững nằm trên ba volume: `mongo_data` (MongoDB), `mediator_state` (hàng đợi SQLite và log) và `search_index` (chỉ mục Lucene). MongoDB có healthcheck; `storage` và `search` chỉ khởi động khi MongoDB đã sẵn sàng. Lược đồ `$jsonSchema` được nạp tự động khi volume MongoDB còn mới.

Listing 6.1: Chạy hệ thống

```
docker compose up -d --build            # khởi động toàn bộ
python3 feed.py                         # nạp 101 461 url mầm
curl localhost:5200/status              # theo dõi hàng đợi
curl localhost:2222/stats               # số bản ghi từng collection
curl -X POST localhost:7700/reindex     # dựng lại chỉ mục sau khi crawl
```

Muốn đổi số bản `downloader` thì sửa hai chỗ cho khớp nhau: `deploy.replicas` trong `compose.yaml` và danh sách `downloaders` trong cấu hình của `mediator`. Các tham số vận hành của `mediator` (khoảng nghỉ theo host 2 giây, hạn ping 2 giây, hạn load 10 giây, thời gian cho node nghỉ 30 giây, tối đa 5 lần thử, chu kỳ crawl lại) nằm trong một tệp YAML; cấu hình sai thì dịch vụ báo lý do và không khởi động.

6.2 Kiểm thử

Mỗi dịch vụ có bộ kiểm thử tự động riêng (Bảng 6.1).

Ngoài ra `mediator` có một `downloader` giả (`demo/fake_fetcher.py`) trả lời đúng giao thức và chỉ ghi log các url nhận được. Nó dùng để chạy thử `mediator` trên một máy và làm mã mẫu cho bên viết `downloader`.

Bảng 6.1: Kiểm thử tự động

Dịch vụ	Số ca	Nội dung chính
`mediator`	65	Chuẩn hoá url, phân loại, lập lịch lại, xoay vòng node, hàng đợi SQLite, API; tích hợp với `downloader` giả qua HTTP; kiểm thử kiến trúc phân lớp (pytest).
`reader`	28	Chọn khối nội dung, văn bản liên kết theo câu, đọc tệp bằng Tika, mã hoá url (JUnit 5).
`storage`	15	Kiểm tra đầu vào, chọn collection, upsert, thống kê (JUnit 5).
`search`	22	Analyzer, PageRank, độ hiếm và ngưỡng thung lũng, loại trùng, profile, API HTTP, nguồn JSON, độ đo đánh giá (JUnit 5).
`downloader`	1	Giao thức `/ping`, `/load`, trạng thái bận (`node:test`).

Chương 7

Hạn chế, hướng phát triển và kết luận

7.1 Hạn chế

- **Chưa dùng văn bản liên kết khi xếp hạng.** `reader` đã bóc được câu chứa liên kết, nhưng collection `graph` chỉ lưu cặp url nên máy tìm kiếm chưa dùng được anchor text, vốn là tín hiệu mạnh cho truy vấn tìm trang cụ thể.
- **Chưa tách từ tiếng Việt.** Bigram âm tiết bắt được phần lớn từ ghép hai âm tiết nhưng không xác định được ranh giới từ, và cụm đảo thứ tự ("phí học") không khớp bigram.
- **PageRank coi mọi liên kết như nhau.** Liên kết menu và chân trang lặp ở mọi trang làm tăng PageRank của trang gốc các website; đồ thị cũng chỉ gồm các trang đã crawl.
- **Loại trùng chỉ bắt trùng hoàn toàn.** Hai trang khác nhau một dòng ngày đăng thì không bị coi là trùng.
- **Phát hiện rác phụ thuộc vào phân bố.** Rác quá ít để thành một cụm riêng trong histogram thì không bị phát hiện.
- **Phải dựng lại toàn bộ chỉ mục.** Hiện chỉ có thể thêm lẻ từng url; trang thêm lẻ chưa có PageRank và chưa qua lọc rác cho đến lần dựng lại kế tiếp.
- **Ảnh chỉ tìm được theo tên tệp.** Không có OCR và chưa dùng `alt` của ảnh làm nội dung.
- **Phát hiện bài mới còn phụ thuộc cấu trúc website.** Cơ chế duyệt trang danh mục cần biết quy tắc đánh số trang của từng website, và sitemap không phải nơi nào cũng đầy đủ; bài mới nằm ngoài cả hai có thể phải chờ đến chu kỳ crawl lại thông thường.
- **Dữ liệu crawl lệch theo host.** Giãn cách theo host làm host nhiều trang chiếm phần lớn dữ liệu; chưa có hạn mức số trang theo host hay ưu tiên theo độ quan trọng (ví dụ tính PageRank từng phần ngay lúc crawl).

- **Đánh giá chưa xong.** Nhãn liên quan chưa gán đủ nên chưa có số liệu định lượng.

7.2 Hướng phát triển

- Lưu văn bản liên kết vào `graph` và index thành trường `anchor` riêng cho tài liệu đích.
- Tách từ tiếng Việt (pyvi, VnCoreNLP) cho trường nội dung, giữ bigram làm trường phụ.
- Phát hiện trang gần trùng bằng SimHash hoặc MinHash; giảm trọng số các liên kết điều hướng lặp khi tính PageRank.
- Cập nhật chỉ mục từng phần theo `updatedAt`, tính lại PageRank định kỳ.
- Gợi ý khi gõ và sửa chính tả, đặc biệt là khôi phục dấu cho truy vấn không dấu.
- OCR cho ảnh và PDF dạng ảnh quét.
- Gán nhãn xong, công bố P@10 và nDCG@10, rồi dùng chúng để chỉnh trọng số trường, tham số b và `PR_BOOST`.

7.3 Kết luận

Nhóm đã xây dựng một máy tìm kiếm hoàn chỉnh cho miền `hust.edu.vn`, từ thu thập đến xếp hạng và đánh giá. Bộ crawler gồm năm dịch vụ đã thu thập hơn 28 nghìn trang web, gần 4 700 tài liệu và 9 600 ảnh từ 95 website của trường, với hàng đợi bền vững, khoảng nghỉ theo host, crawl lại định kỳ, phát hiện bài mới qua sitemap và trang danh mục, và tự tiếp tục khi một khâu gặp lỗi. Máy tìm kiếm xử lý được các đặc thù của tiếng Việt (chuẩn hoá Unicode, cụm hai âm tiết, truy vấn không dấu), kết hợp BM25 nhiều trường với cụm từ và PageRank trong mô hình hai bước, và có cơ chế lọc rác, loại trùng không cần đặt ngưỡng bằng tay. Bốn profile xếp hạng cùng công cụ pooling và nDCG@10 là nền để đo và cải tiến chất lượng xếp hạng một cách có căn cứ khi việc gán nhãn hoàn tất.

Tài liệu tham khảo

[1] C. D. Manning, P. Raghavan, H. Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[2] S. Robertson, H. Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[3] L. Page, S. Brin, R. Motwani, T. Winograd. The PageRank Citation Ranking: Bringing Order to the Web. Technical report, Stanford InfoLab, 1999.

[4] J. M. S. Prewitt, M. L. Mendelsohn. The Analysis of Cell Images. *Annals of the New York Academy of Sciences*, 128(3):1035–1053, 1966.

[5] K. Järvelin, J. Kekäläinen. Cumulated Gain-Based Evaluation of IR Techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002.

[6] Apache Software Foundation. Apache Lucene 10. `https://lucene.apache.org/`

[7] Apache Software Foundation. Apache Tika. `https://tika.apache.org/`

[8] Unicode Consortium. Unicode Standard Annex #29: Unicode Text Segmentation. `https://unicode.org/reports/tr29/`

[9] Google. Puppeteer: Headless Chrome Node.js API. `https://pptr.dev/`